INTRODUCTION

Definition of a circular linked list

- Circular linked lists are a variation where the last node points back to the first, forming a loop.

Overview of linked list structure

- Similar to singly linked lists, but the last node's next pointer points to the first node, creating a circular structure.

Purpose and common use cases

- Useful in scenarios where continuous cyclic traversal is needed, such as scheduling algorithms.

CHARACTERISTICS OF A CIRCULAR LINKED LIST

Circular structure

- The last node's next pointer points to the first node, forming a closed loop.

Dynamic size

- Like other linked lists, circular linked lists can dynamically grow or shrink.

Nodes containing data and a reference to the next node

- Each node contains data and a pointer to the next node, similar to singly linked lists.

WHY USE CIRCULAR LINKED LISTS?

Continuous cyclic traversal

- Ideal for scenarios requiring continuous cyclic traversal, like in circular buffers or scheduling.

Efficient for certain algorithms

- More efficient for certain algorithms where looping back to the beginning is a natural operation.

Applications in scenarios with cyclic data patterns

- Useful when dealing with data that repeats in a cyclic manner.

BASIC LINKED LIST OPERATIONS FOR CIRCULAR LINKED LISTS

Insertion: insertFront and insertEnd

- Similar to singly linked lists, but now considering the circular structure.

Deletion: deleteFront and deleteEnd

- Adjustments for the circular structure when removing nodes.

Search

- Searching is performed similarly to singly linked lists.

EXAMPLE: CIRCULAR QUEUE IMPLEMENTATION

Example of circular linked list

```
// Initialize the Nodes.
Node one = new Node(30);
Node two = new Node(50);
Node three = new Node(90);

// Connect nodes
one.next = two;
two.next = three;
three.next = one;
```

INSERTIONS

Insertion at the end of the list:

- For insertion at the end of the circular linked list, we will store the address of the head node to the new node and make the last node a new node.

INSERTIONS

Insertion at the given position:

- For insertion at the given position, we will traverse the circular linked list and point the next of the new node to the next node.

DELETIONS

Delete the first node of the list:

- To delete the first node, transfer the head node to the next node and free the memory of the first node.

DELETION

Delete the last node of the list:

- To delete the last node of a circular list, find the second last node, change its pointer to the next node, and free the memory of the last node.

DELETIONS

Delete the node from any position:

- To delete the node from a given position, you need to traverse the list, store the address of the previous node, and point the next of the current node to the next node.

---

complete C program for Circular Linked List (CLL) with common operations:

- Create node
- Insert at beginning
- Insert at end
- Insert at position
- Delete from beginning
- Delete from end
- Delete from position
- Search element
- Display list
- Count nodes

C/C++

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *last = NULL;

// Create a new node
struct Node* createNode(int value)
{
    struct Node *newNode = (struct Node*)malloc(sizeof(struct
Node));
    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        exit(1);
    }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

// Display circular linked list
void display()
{
    if (last == NULL)
    {
        printf("Circular Linked List is empty.\n");
        return;
    }

    struct Node *temp = last->next; // head
```

```c
    printf("Circular Linked List: ");
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != last->next);
    printf("\n");
}

// Count nodes
int countNodes()
{
    if (last == NULL)
        return 0;

    int count = 0;
    struct Node *temp = last->next;
    do
    {
        count++;
        temp = temp->next;
    } while (temp != last->next);

    return count;
}

// Insert at beginning
void insertAtBeginning(int value)
{
    struct Node *newNode = createNode(value);

    if (last == NULL)
    {
        last = newNode;
        last->next = last;
    }
```

```
    else
    {
        newNode->next = last->next;
        last->next = newNode;
    }

    printf("%d inserted at beginning.\n", value);
}

// Insert at end
void insertAtEnd(int value)
{
    struct Node *newNode = createNode(value);

    if (last == NULL)
    {
        last = newNode;
        last->next = last;
    }
    else
    {
        newNode->next = last->next;
        last->next = newNode;
        last = newNode;
    }

    printf("%d inserted at end.\n", value);
}

// Insert at position
void insertAtPosition(int value, int pos)
{
    int n = countNodes();

    if (pos < 1 || pos > n + 1)
    {
```

```
        printf("Invalid position.\n");
        return;
    }

    if (pos == 1)
    {
        insertAtBeginning(value);
        return;
    }

    if (pos == n + 1)
    {
        insertAtEnd(value);
        return;
    }

    struct Node *newNode = createNode(value);
    struct Node *temp = last->next;
    int i;

    for (i = 1; i < pos - 1; i++)
    {
        temp = temp->next;
    }

    newNode->next = temp->next;
    temp->next = newNode;

    printf("%d inserted at position %d.\n", value, pos);
}

// Delete from beginning
void deleteFromBeginning()
{
    if (last == NULL)
    {
```

```
        printf("List is empty.\n");
        return;
    }

    struct Node *head = last->next;

    if (last == head)
    {
        printf("%d deleted from beginning.\n", head->data);
        free(head);
        last = NULL;
    }
    else
    {
        last->next = head->next;
        printf("%d deleted from beginning.\n", head->data);
        free(head);
    }
}

// Delete from end
void deleteFromEnd()
{
    if (last == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    struct Node *head = last->next;

    if (last == head)
    {
        printf("%d deleted from end.\n", last->data);
        free(last);
        last = NULL;
```

```
        return;
    }

    struct Node *temp = head;
    while (temp->next != last)
    {
        temp = temp->next;
    }

    temp->next = last->next;
    printf("%d deleted from end.\n", last->data);
    free(last);
    last = temp;
}

// Delete from position
void deleteFromPosition(int pos)
{
    int n = countNodes();

    if (last == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    if (pos < 1 || pos > n)
    {
        printf("Invalid position.\n");
        return;
    }

    if (pos == 1)
    {
        deleteFromBeginning();
        return;
```

```
    }

    if (pos == n)
    {
        deleteFromEnd();
        return;
    }

    struct Node *temp = last->next;
    struct Node *delNode;
    int i;

    for (i = 1; i < pos - 1; i++)
    {
        temp = temp->next;
    }

    delNode = temp->next;
    temp->next = delNode->next;

    printf("%d deleted from position %d.\n", delNode->data, pos);
    free(delNode);
}

// Search element
void searchElement(int key)
{
    if (last == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    struct Node *temp = last->next;
    int pos = 1;
    int found = 0;
```

```
    do
    {
        if (temp->data == key)
        {
            printf("%d found at position %d.\n", key, pos);
            found = 1;
            break;
        }
        temp = temp->next;
        pos++;
    } while (temp != last->next);

    if (!found)
    {
        printf("%d not found in the list.\n", key);
    }
}

// Free entire list
void freeList()
{
    if (last == NULL)
        return;

    struct Node *head = last->next;
    struct Node *temp = head;
    struct Node *nextNode;

    do
    {
        nextNode = temp->next;
        free(temp);
        temp = nextNode;
    } while (temp != head);
```

```
    last = NULL;
}

int main()
{
    int choice, value, pos, key;

    while (1)
    {
        printf("\n--- Circular Linked List Menu ---\n");
        printf("1. Insert at beginning\n");
        printf("2. Insert at end\n");
        printf("3. Insert at position\n");
        printf("4. Delete from beginning\n");
        printf("5. Delete from end\n");
        printf("6. Delete from position\n");
        printf("7. Search element\n");
        printf("8. Display list\n");
        printf("9. Count nodes\n");
        printf("10. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insertAtBeginning(value);
                break;

            case 2:
                printf("Enter value: ");
                scanf("%d", &value);
                insertAtEnd(value);
                break;
```

```c
        case 3:
            printf("Enter value: ");
            scanf("%d", &value);
            printf("Enter position: ");
            scanf("%d", &pos);
            insertAtPosition(value, pos);
            break;

        case 4:
            deleteFromBeginning();
            break;

        case 5:
            deleteFromEnd();
            break;

        case 6:
            printf("Enter position: ");
            scanf("%d", &pos);
            deleteFromPosition(pos);
            break;

        case 7:
            printf("Enter element to search: ");
            scanf("%d", &key);
            searchElement(key);
            break;

        case 8:
            display();
            break;

        case 9:
            printf("Total nodes: %d\n", countNodes());
            break;
```

```
            case 10:
                freeList();
                printf("Program exited.\n");
                return 0;

            default:
                printf("Invalid choice.\n");
        }
    }

    return 0;
}
```

Sample operations covered

This code supports:

- `insertAtBeginning()`
- `insertAtEnd()`
- `insertAtPosition()`
- `deleteFromBeginning()`
- `deleteFromEnd()`
- `deleteFromPosition()`
- `searchElement()`
- `display()`
- `countNodes()`

How to run

Save as `cll.c`

Compile:

gcc cll.c -o cll

Run:

./cll

On Windows with GCC:

gcc cll.c -o cll.exe

cll.exe

TIME AND SPACE COMPLEXITY FOR CIRCULAR LINKED LISTS

Time Complexity of Circular Linked List:

- **Insertion Operation:** O(1) or O(N)
- **Deletion Operation:** O(1)

Space Complexity of Circular Linked List:

- **Insertion Operation:** O(1)
- **Deletion Operation:** O(1)

TYPES OF CIRCULAR LINKED LIST

1. Circular Singly Linked List

- Each node contains:
    - Data
    - Pointer to next node
- The last node points back to the first node (head), forming a loop.

2. Circular Doubly Linked List

- Each node contains:
    - Data
    - Pointer to next node
    - Pointer to previous node
- The last node connects back to the first node, and the first node points to the last node, forming a two-way circular structure.

COMMON MISTAKES AND PITFALLS IN CIRCULAR LINKED LISTS

1. Ensuring proper termination conditions

- Emphasizes the need for correct stopping conditions to avoid infinite loops.

2. Handling edge cases in circular structure

- Important to manage boundary conditions such as:
    - Empty list
    - Single node list
    - Insertion/deletion at head or last

3. Failure to free allocated memory

- Highlights the importance of freeing memory to prevent memory leaks.
